

TA Activity -2

Name - Avdhut kale

Roll no -43

Name - Kartik Bramhat

Roll no - 06

✓ Mini Project Title

Skin Disease Classification using Deeplearning

✓ 1. Data Loading

- Mount Google Drive
- Load dataset CSV file
- Read image paths

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

✓ 2. Data Preprocessing

- Create image paths using function
- Remove invalid/missing images
- Split dataset:
 - Training set (80%)
 - Testing set (20%)

```
import os

DATA_DIR = "/content/drive/MyDrive/Colab Notebooks/Annd1"

IMG_DIR_1 = os.path.join(DATA_DIR, "HAM10000_images_part_1")
IMG_DIR_2 = os.path.join(DATA_DIR, "HAM10000_images_part_2")

CSV_PATH = os.path.join(DATA_DIR, "HAM10000_metadata.csv")
```

```
import numpy as np
import pandas as pd
import tensorflow as tf
```

```
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

3. Data Augmentation

- Rescale images
- Rotation, zoom, flip
- Use ImageDataGenerator

```
df = pd.read_csv(CSV_PATH)
df = df[['image_id', 'dx']]

print(df.head())
print(df['dx'].value_counts())
```

```
      image_id  dx
0  ISIC_0027419  bkl
1  ISIC_0025030  bkl
2  ISIC_0026769  bkl
3  ISIC_0025661  bkl
4  ISIC_0031633  bkl
dx
nv      6705
mel     1113
bkl     1099
bcc      514
akiec   327
vasc    142
df      115
Name: count, dtype: int64
```

```
import os

def get_path(image_id):
    path1 = os.path.join(IMG_DIR_1, image_id + ".jpg")
    path2 = os.path.join(IMG_DIR_2, image_id + ".jpg")

    if os.path.exists(path1):
        return path1
    elif os.path.exists(path2):
        return path2
    else:
        return None # Return None if the image is not found in either directory

df['path'] = df['image_id'].apply(get_path)
```

```
print(f"DataFrame shape before filtering: {df.shape}")
df = df.dropna(subset=['path'])
print(f"DataFrame shape after filtering: {df.shape}")
```

```
DataFrame shape before filtering: (10015, 3)
DataFrame shape after filtering: (7018, 3)
```

```
print(f"DataFrame shape before filtering: {df.shape}")
df = df.dropna(subset=['path'])
print(f"DataFrame shape after filtering: {df.shape}")
```

```
DataFrame shape before filtering: (7018, 3)
DataFrame shape after filtering: (7018, 3)
```

```
train_data = train_gen.flow_from_dataframe(
    dataframe=train_df,
    x_col="path",
    y_col="dx",
    target_size=(224,224),
    class_mode="categorical",
    batch_size=32
)

test_data = test_gen.flow_from_dataframe(
    dataframe=test_df,
    x_col="path",
    y_col="dx",
    target_size=(224,224),
    class_mode="categorical",
    batch_size=32
)
```

```
Found 5614 validated image filenames belonging to 7 classes.
Found 1404 validated image filenames belonging to 7 classes.
```

```
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
```

```
train_gen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

test_gen = ImageDataGenerator(rescale=1./255)
```

```
train_data = train_gen.flow_from_dataframe(
    dataframe=train_df,
    x_col="path",
    y_col="dx",
    target_size=(224,224),
    class_mode="categorical",
    batch_size=32
)

test_data = test_gen.flow_from_dataframe(
    dataframe=test_df,
    x_col="path",
    y_col="dx",
    target_size=(224,224),
    class_mode="categorical",
    batch_size=32
)
```

```
and 5639 validated image filenames belonging to 7 classes.
sr/local/lib/python3.12/dist-packages/keras/src/legacy/preprocessing/image.py:918: UserWarning: Four
warnings.warn(
and 1379 validated image filenames belonging to 7 classes.
sr/local/lib/python3.12/dist-packages/keras/src/legacy/preprocessing/image.py:918: UserWarning: Four
warnings.warn(
```

✓ 4. Model Development

- Use **MobileNetV2 (Pre-trained model)**
- Freeze base layers
- Add custom layers:
 - Global Average Pooling
 - Dense Layer
 - Dropout
 - Output Layer (Softmax)

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model

base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)

# ✅ FIX HERE
output = Dense(len(train_data.class_indices), activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
```

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    train_data,
    validation_data=test_data,
    epochs=10
)
```

```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
```

```
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
```

```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()
```

```
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix

# Predictions
y_pred = model.predict(test_data)
y_pred_classes = np.argmax(y_pred, axis=1)

# True labels
y_true = test_data.classes

print(classification_report(y_true, y_pred_classes))
print(confusion_matrix(y_true, y_pred_classes))
```

```
import cv2

def predict_image(img_path):
    img = cv2.imread(img_path)
    img = cv2.resize(img, (224,224))
    img = img / 255.0
    img = np.reshape(img, (1,224,224,3))

    pred = model.predict(img)
    class_idx = np.argmax(pred)

    class_labels = list(train_data.class_indices.keys())
    return class_labels[class_idx]

# Example
print(predict_image("/content/drive/MyDrive/Colab Notebooks/Annd1/HAM10000_images_part_1/ISIC_002"))
```

```
model.save("/content/drive/MyDrive/skin_disease_model.h5")
```

```
def get_path(image_id):
    path1 = os.path.join(IMG_DIR_1, image_id + ".jpg")
    path2 = os.path.join(IMG_DIR_2, image_id + ".jpg")

    if os.path.exists(path1):
        return path1
    else:
        return path2

df['path'] = df['image_id'].apply(get_path)
```

```
print(df.head())
print(df['dx'].value_counts())
```

```

      image_id  dx                                     path
0  ISIC_0027419  bkl  /content/drive/MyDrive/Colab Notebooks/Anndl/H...
1  ISIC_0025030  bkl  /content/drive/MyDrive/Colab Notebooks/Anndl/H...
2  ISIC_0026769  bkl  /content/drive/MyDrive/Colab Notebooks/Anndl/H...
3  ISIC_0025661  bkl  /content/drive/MyDrive/Colab Notebooks/Anndl/H...
4  ISIC_0031633  bkl  /content/drive/MyDrive/Colab Notebooks/Anndl/H...
dx
nv      6705
mel     1113
bkl     1099
bcc       514
akiec    327
vasc     142
df       115
Name: count, dtype: int64
```

```

train_gen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

test_gen = ImageDataGenerator(rescale=1./255)

print("ImageDataGenerators 'train_gen' and 'test_gen' initialized.")
```

ImageDataGenerators 'train_gen' and 'test_gen' initialized.

```

train_data = train_gen.flow_from_dataframe(
    dataframe=train_df,
    x_col="path",
    y_col="dx",
    target_size=(224,224),
    class_mode="categorical",
    batch_size=32
)

test_data = test_gen.flow_from_dataframe(
    dataframe=test_df,
    x_col="path",
    y_col="dx",
    target_size=(224,224),
    class_mode="categorical",
    batch_size=32
)
```

Found 5639 validated image filenames belonging to 7 classes.
/usr/local/lib/python3.12/dist-packages/keras/src/legacy/preprocessing/image.py:918: UserWarning:
warnings.warn(
Found 1379 validated image filenames belonging to 7 classes.
/usr/local/lib/python3.12/dist-packages/keras/src/legacy/preprocessing/image.py:918: UserWarning:
warnings.warn(

```

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model
```

```

base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)

# ✅ FIX HERE
output = Dense(len(train_data.class_indices), activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_data_format_224x224_tf_dim_ordering_tf_data_format_192.h5 9406464/9406464 ————— 0s 0us/step

```

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
print("Model compiled successfully.")

```

Model compiled successfully.

✓ 6. Model Training

- Train model using training data
- Validate using test data
- Track accuracy and loss

```

history = model.fit(
    train_data,
    validation_data=test_data,
    epochs=10
)
print("Model training initiated.")

```

```

Epoch 1/10
176/176 ————— 1871s 11s/step - accuracy: 0.6936 - loss: 0.9547 - val_accuracy: 0.71
Epoch 2/10
176/176 ————— 477s 3s/step - accuracy: 0.7243 - loss: 0.8104 - val_accuracy: 0.7236
Epoch 3/10
176/176 ————— 469s 3s/step - accuracy: 0.7342 - loss: 0.7600 - val_accuracy: 0.7472
Epoch 4/10
176/176 ————— 460s 3s/step - accuracy: 0.7419 - loss: 0.7388 - val_accuracy: 0.7372
Epoch 5/10
176/176 ————— 502s 3s/step - accuracy: 0.7439 - loss: 0.7284 - val_accuracy: 0.7479
Epoch 6/10
176/176 ————— 467s 3s/step - accuracy: 0.7547 - loss: 0.7030 - val_accuracy: 0.7479
Epoch 7/10
176/176 ————— 477s 3s/step - accuracy: 0.7576 - loss: 0.6868 - val_accuracy: 0.7493

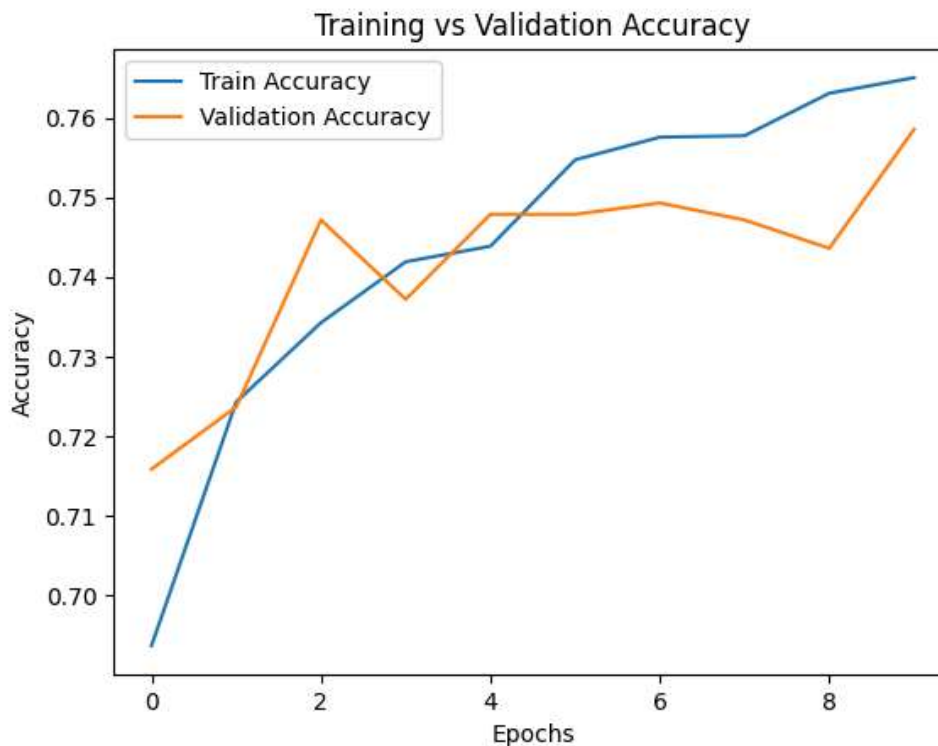
```

Epoch 8/10
176/176 ————— 464s 3s/step - accuracy: 0.7577 - loss: 0.6852 - val_accuracy: 0.7472
Epoch 9/10
176/176 ————— 472s 3s/step - accuracy: 0.7631 - loss: 0.6639 - val_accuracy: 0.7436
Epoch 10/10
176/176 ————— 474s 3s/step - accuracy: 0.7651 - loss: 0.6612 - val_accuracy: 0.7585
Model training initiated.

```
import matplotlib.pyplot as plt

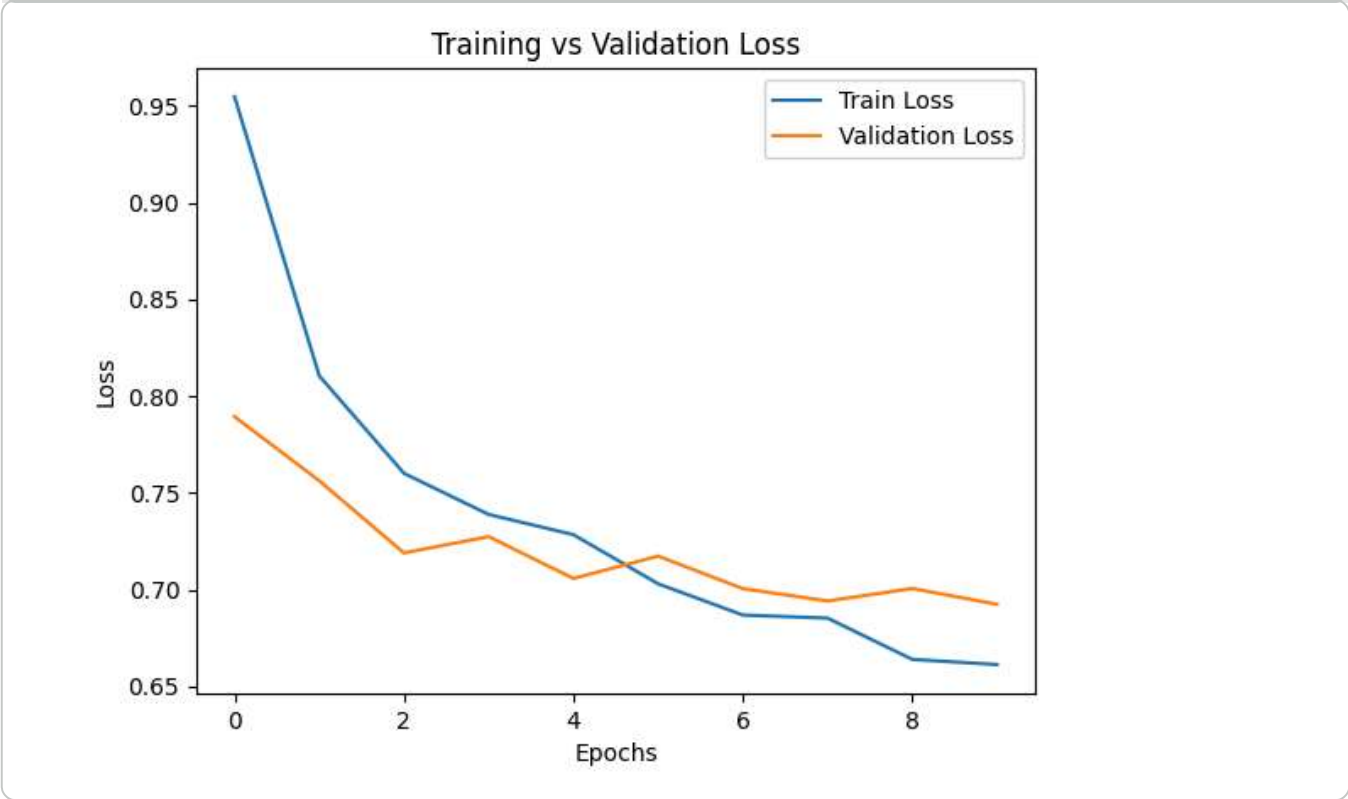
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
```



```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()
```



```
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix

# Predictions
y_pred = model.predict(test_data)
y_pred_classes = np.argmax(y_pred, axis=1)

# True labels
y_true = test_data.classes

print(classification_report(y_true, y_pred_classes))
print(confusion_matrix(y_true, y_pred_classes))
```

44/44

80s 2s/step

	precision	recall	f1-score	support
0	0.05	0.04	0.04	49
1	0.11	0.08	0.09	89
2	0.10	0.11	0.11	170
3	0.00	0.00	0.00	14
4	0.06	0.04	0.05	119
5	0.68	0.73	0.70	944
6	0.00	0.00	0.00	19
accuracy			0.52	1404
macro avg	0.14	0.14	0.14	1404
weighted avg	0.48	0.52	0.50	1404

```
[[ 2  3  4  0  2 38  0]
 [ 3  7 10  0  5 64  0]
 [ 8  5 19  0  9 127 2]
 [ 1  3  2  0  1  7  0]
 [ 5  7 20  0  5  80 2]
 [25 38 124 0 59 692 6]
 [ 0  0  2  0  1  16  0]]
```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is undefined for samples with no predicted labels. The F-score and F1-score will be NaN.

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is undefined because no predicted samples belong to this class.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is undefined because no predicted samples belong to this class.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
import cv2

def predict_image(img_path):
    img = cv2.imread(img_path)
    img = cv2.resize(img, (224,224))
    img = img / 255.0
    img = np.reshape(img, (1,224,224,3))

    pred = model.predict(img)
    class_idx = np.argmax(pred)

    class_labels = list(train_data.class_indices.keys())
    return class_labels[class_idx]

# Example
print(predict_image("/content/drive/MyDrive/Colab Notebooks/Anndl/HAM10000_images_part_1/ISIC_0024"))
```

1/1  1s 1s/step
bkl

```
model.save("/content/drive/MyDrive/skin_disease_model.keras")
print("Model saved to /content/drive/MyDrive/skin_disease_model.keras")
```

Model saved to /content/drive/MyDrive/skin_disease_model.keras

```
loss, acc = model.evaluate(test_data)
print("Test Accuracy:", acc)
```

44/44  78s 2s/step - accuracy: 0.7585 - loss: 0.6924
Test Accuracy: 0.7585470080375671

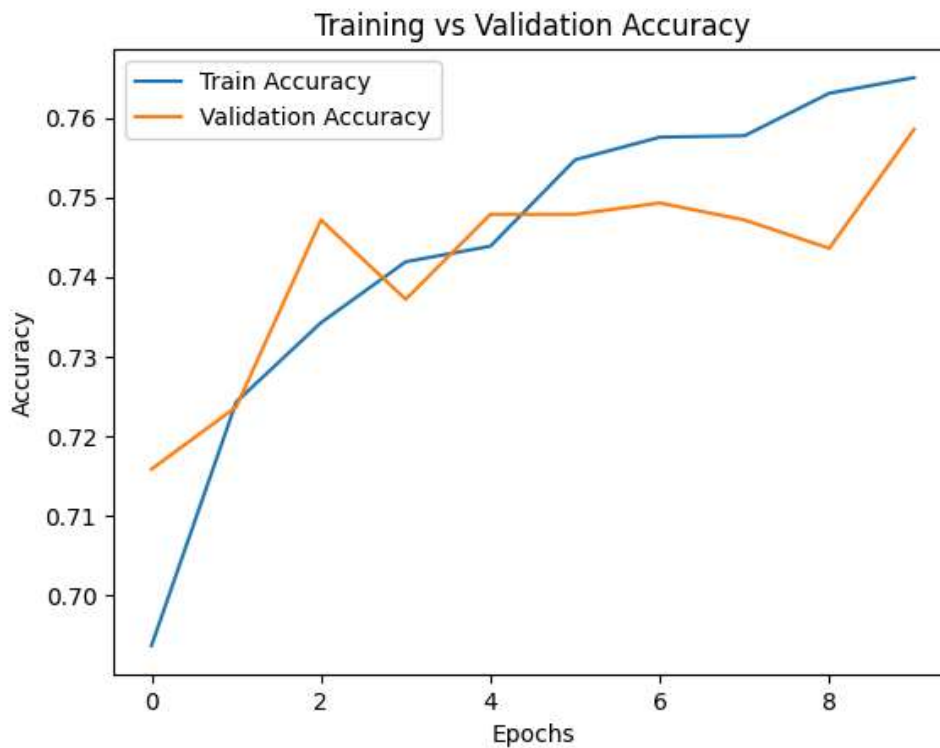
✓ 8. Prediction

- Input single image
- Preprocess image
- Predict disease class

```
import matplotlib.pyplot as plt

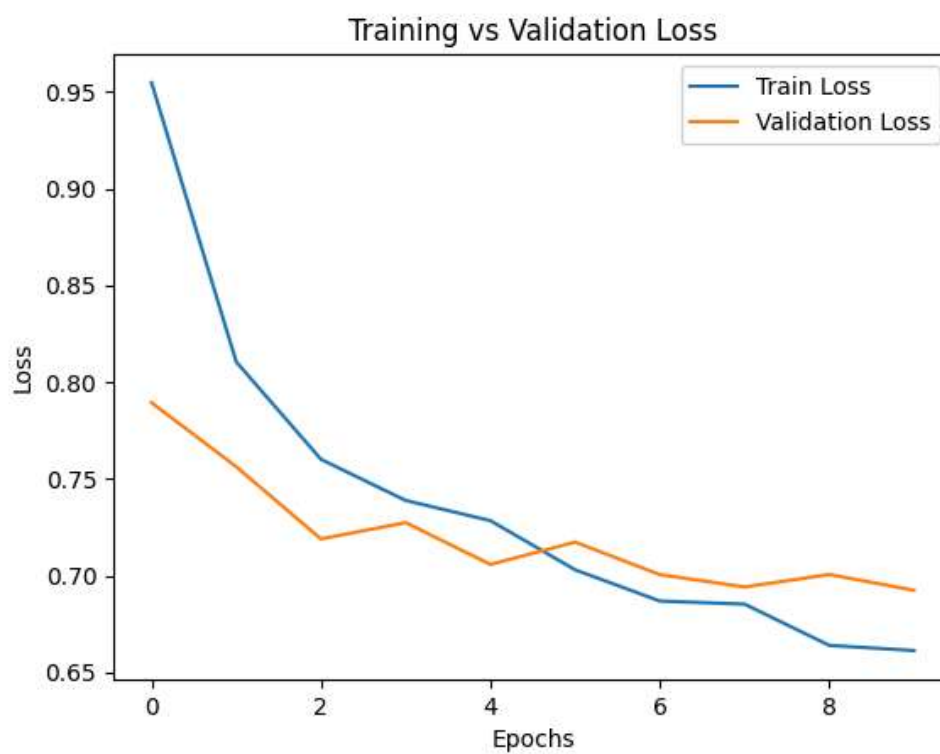
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
```



```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()
```



```
import cv2

def predict_image(img_path):
    img = cv2.imread(img_path)
    img = cv2.resize(img, (224,224))
    img = img / 255.0
    img = np.reshape(img, (1,224,224,3))

    pred = model.predict(img)
    class_idx = np.argmax(pred)

    class_labels = list(train_data.class_indices.keys())
    return class_labels[class_idx]

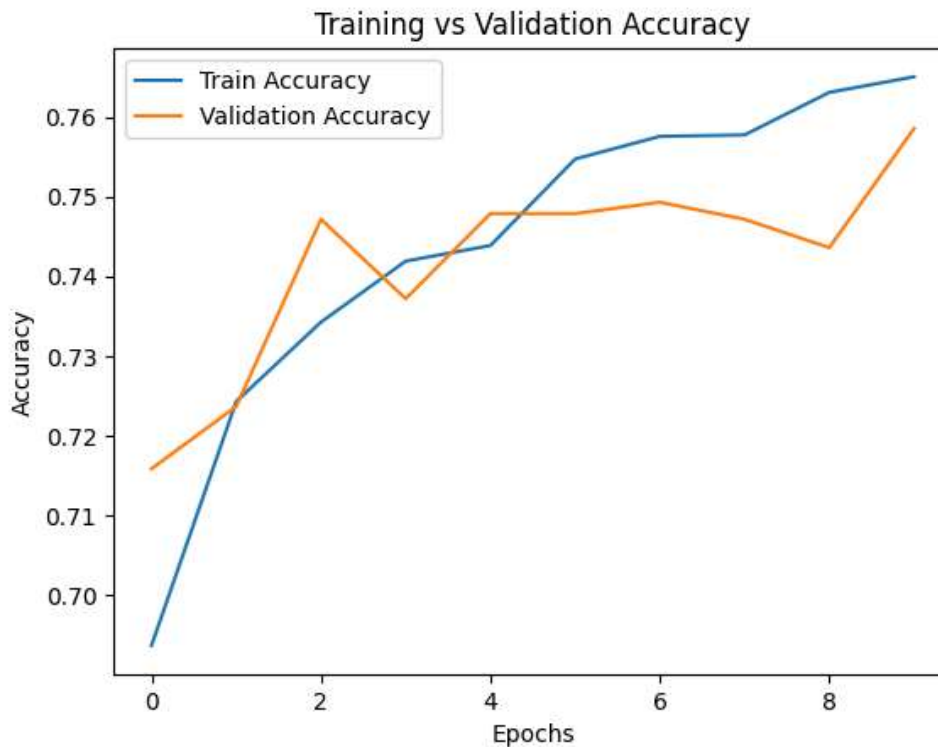
# Example
print(predict_image("/content/drive/MyDrive/Colab Notebooks/Annd1/HAM10000_images_part_1/ISIC_002
```

1/1 ————— 0s 92ms/step
bkl

```
import matplotlib.pyplot as plt

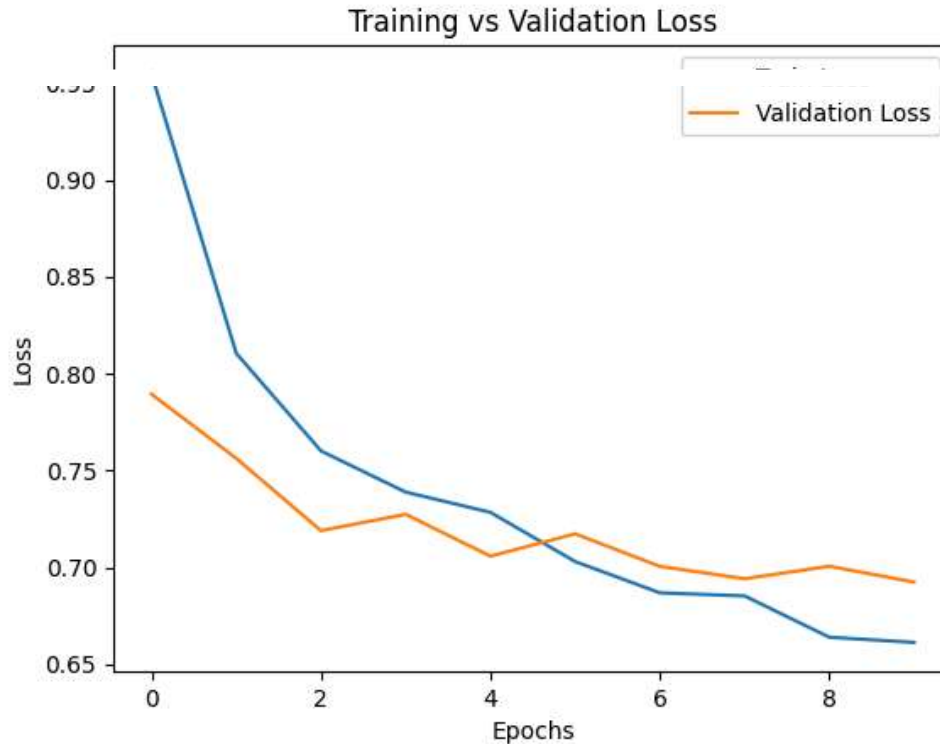
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
```



```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()
```



Instructions

Execute cell `5XUcqg5Dh\_pp` to generate the classification report and confusion matrix for the model

```
model.save("/content/drive/MyDrive/skin_disease_model.h5")
print("Model saved to /content/drive/MyDrive/skin_disease_model.h5")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model`. Model saved to /content/drive/MyDrive/skin_disease_model.h5